



OFFICIAL DEVELOPER REFERENCE

ClickPesa PHP & Laravel SDK

Complete Integration Guide & API Reference Manual

UBITECH SOLUTIONS LIMITED

Professional Software Engineering & Digital Payment Integrations

 **Phone / Support: +255 766 192 332**

Repository: github.com/ubitecholutionsltd/clickpesa-laravel-package

Release Version: 1.0.0 • September 2026

1. Executive Summary & Architecture

The **ClickPesa PHP & Laravel SDK** is an enterprise-grade library engineered by **UBITECH SOLUTIONS LIMITED** to provide frictionless integration with ClickPesa's collection and disbursement APIs across Tanzania and East Africa.

Core Principles

- **Universal Compatibility (Any PHP Application):** Standalone core with zero framework lock-in. Compatible with PHP 8.1, 8.2, and 8.3 in vanilla PHP, Symfony, WordPress, or CLI tools.
- **First-Class Laravel Integration:** Out-of-the-box auto-discovery, publishable configuration, static `ClickPesa` Facade, native Cache token persistence, Webhook routing macro, and dispatched Laravel events.
- **Zero-Maintenance Authentication:** Automatic token management that handles issuance, in-memory/cache storage, and preemptive refresh before the 1-hour expiration limit.
- **Compliant Checksum Engine:** Built-in canonicalization and HMAC-SHA256 signing for outgoing requests and timing-safe webhook verification.
- **Built-in Testing Mock:** `ClickPesa::fake()` and assertion helpers for instant unit and integration testing without reaching production endpoints.

Note on ClickPesa Environment: ClickPesa operates directly against live production with small amounts and pre-KYC tier limits. There is no separate sandbox domain; all calls target <https://api.clickpesa.com/third-parties>.

2. Installation & Configuration

Installing with Composer

```
composer require clickpesa/clickpesa-laravel-sdk
```

Publishing Laravel Configuration

```
php artisan vendor:publish --tag=clickpesa-config
```

This generates `config/clickpesa.php`. Set your credentials in your `.env` file:

```
# .env
CLICKPESA_CLIENT_ID=your_client_id_here
CLICKPESA_API_KEY=your_api_key_here
CLICKPESA_CHECKSUM_KEY=your_checksum_secret_here
CLICKPESA_CHECKSUM_ENABLED=true
CLICKPESA_TIMEOUT=30
CLICKPESA_TOKEN_TTL_MARGIN=300
```

3. Authentication & Security Engine

Automatic Token Management

ClickPesa requires a JWT bearer token generated via `POST /generate-token` using the `client-id` and `api-key` request headers. The token is valid for exactly **1 hour**.

The SDK encapsulates this logic inside `ClickPesa\Auth\TokenManager`. On every authenticated request:

1. The SDK checks if a valid token exists in cache (in-memory for standalone PHP, or Laravel Cache in Laravel).
2. If expired or missing, it automatically calls ClickPesa's `/generate-token` API and stores the token with a 5-minute safety buffer.
3. If an unexpected 401 response occurs, it proactively flushes the token and transparently retries the call once.

Canonicalization & Checksum Engine

When checksum is enabled, ClickPesa enforces an HMAC-SHA256 signature calculated on a canonicalized JSON payload:

1. **Alphabetical Key Sorting:** Recursively sorts all associative array/object keys alphabetically at every level.
2. **Preserving Sequential Arrays:** Numeric lists maintain their original order while sorting nested objects.
3. **Excluding Checksum Fields:** Automatically strips `checksum` and `checksumMethod` fields before calculation.
4. **Compact JSON Serialization:** Serialized using `json_encode($data, JSON_UNESCAPED_SLASHES)`.
5. **HMAC-SHA256:** Signed using your secret checksum key.

```
use ClickPesa\Security\Checksum;

// Generate outgoing checksum
$hash = Checksum::generate($secretKey, $payload);

// Timing-safe verification for incoming webhooks
$isValid = Checksum::verify($secretKey, $payload, $receivedChecksum);
```

4. Payment Collections API

1. Mobile USSD-PUSH

Collect payments instantly by pushing USSD authorization prompts to customer handsets on Vodacom M-Pesa, Tigo-Pesa, Airtel Money, or Halopesa.

Preview Fees & Available Networks

```
use ClickPesa\Laravel\Facades\ClickPesa;

$preview = ClickPesa::ussdPush()->preview([
    'amount'          => '10000',
    'currency'         => 'TZS',
    'orderReference'  => 'ORD-1001',
    'phoneNumber'     => '255712345678',
    'fetchSenderDetails' => true,
]);
```

Initiate USSD-PUSH Prompt

```
$response = ClickPesa::ussdPush()->initiate([
    'amount'          => '10000',
    'currency'         => 'TZS',
    'orderReference'  => 'ORD-1001',
    'phoneNumber'     => '255712345678',
]);

// Returns:
// [
//   'id' => 'PUSH12345',
//   'status' => 'PROCESSING',
//   'channel' => 'TIGO-PESA',
//   'orderReference' => 'ORD-1001',
//   'collectedAmount' => '10000'
// ]
```

2. Card Payments

Accept payments from Visa, MasterCard, UnionPay, and American Express.

```

$card = ClickPesa::cardPayments()->initiate([
    'amount'      => '50',
    'currency'     => 'USD',
    'orderReference' => 'CARD-ORD-902',
    'customer' => [
        'fullName'   => 'Jane Doe',
        'email'       => 'jane@example.com',
        'phoneNumber' => '255712345678',
    ],
]);

$redirectLink = $card['cardPaymentLink'];

```

3. Hosted Checkout Links

Generate secure, hosted checkout pages supporting itemized orders or fixed totals.

```

$link = ClickPesa::checkoutLinks()->generate([
    'totalPrice'    => '45000',
    'orderReference' => 'INV-2045',
    'orderCurrency' => 'TZS',
    'customerName'  => 'Mathayo John',
    'customerEmail' => 'mathayo@example.com',
    'customerPhone' => '255712345678',
    'description'   => 'Payment for Software License',
]);

$hostedUrl = $link['checkoutLink'];

```

5. BillPay Control Numbers & Direct Debit

BillPay Control Numbers

Generate standard 14-digit Tanzanian control numbers for offline and online customer payments via banks and mobile money wallets.

Create Order Control Number (One-Time Invoice)

```
$orderBill = ClickPesa::billPay()->createOrder([
    'billDescription' => 'Water Bill - September 2026',
    'billAmount'      => 90000,
    'billPaymentMode' => 'ALLOW_PARTIAL_AND_OVER_PAYMENT',
    'billReference'   => 'INV-SEPT-01',
]);
$controlNumber = $orderBill['billPayNumber'];
```

Create Customer Control Number (Reusable Account)

```
$custBill = ClickPesa::billPay()->createCustomer();
$customerControlNo = $custBill['billPayNumber'];
```

Bulk Control Numbers (Up to 50 Items)

```
$bulk = ClickPesa::billPay()->bulkCreateOrders([
    ['billDescription' => 'Invoice 101', 'billAmount' => 15000],
    ['billDescription' => 'Invoice 102', 'billAmount' => 25000],
]);
```

CRDB Direct Debit Mandates

Request recurring direct debit authorizations from customers' CRDB Bank accounts.

```
// Request Mandate Approval
$mandate = ClickPesa::crdbDirectDebit()->requestMandate([
    'accountNumber' => '0150123456700',
    'amount'         => 50000,
    'orderReference' => 'MANDATE-01',
    'startDate'      => '2026-10-01',
    'frequency'      => 'MONTHLY',
]);

// Inspect Mandate Details
$details = ClickPesa::crdbDirectDebit()->getMandate('MANDATE_REQ_ID_123');

// Cancel Active Mandate
ClickPesa::crdbDirectDebit()->cancelMandate(['mandateRequestId' => 'MANDATE_REQ_ID_123']);
```

Querying Payments

```
// Query specific payment attempts by Order Reference
$attempts = ClickPesa::payments()->get('ORD-1001');

// Query with pagination and filtering
$allPayments = ClickPesa::payments()->all([
    'status'      => 'SUCCESS',
    'startDate'   => '2026-09-01',
    'limit'       => 25,
]);
```

6. Disbursements & Payouts API

60-Second Cooldown Warning: ClickPesa enforces a strict 60-second cooldown between payout creation requests per merchant. The SDK automatically catches this and raises a `RateLimitException` containing `$e->getRetryAfterSeconds()`.

1. Mobile Money (MNO) Payouts

Disburse funds directly to Vodacom M-Pesa, Tigo-Pesa, Airtel Money, or Halopesa wallets.

```
// 1. Preview Fee & Balance
$preview = ClickPesa::mobileMoneyPayouts()->preview([
    'amount'      => 10000,
    'currency'     => 'TZS',
    'orderReference' => 'PAY-001',
    'phoneNumber' => '255712345678',
]);

// 2. Disburse Payout
$payout = ClickPesa::mobileMoneyPayouts()->create([
    'amount'      => 10000,
    'currency'     => 'TZS',
    'orderReference' => 'PAY-001',
    'phoneNumber' => '255712345678',
]);
```

2. Bank Transfers

Disburse payouts to any commercial bank account in Tanzania using Account Number and BIC.

```
$bankPayout = ClickPesa::bankPayouts()->create([
    'amount'      => 500000,
    'currency'     => 'TZS',
    'orderReference' => 'BANK-PAY-101',
    'accountNumber' => '0150123456700',
    'accountName'  => 'Acme Trading Ltd',
    'bic'          => 'CORUTZTZ', // CRDB Bank BIC
]);
```

3. Lipa Namba & TanQR Payouts

Send money directly to any merchant Lipa Namba or TanQR code.

```
// 1. Get providers (e.g. Vodacom 503, Airtel 502)
$providers = ClickPesa::lipaNambaPayouts()->providers();

// 2. Send to Lipa Namba
$lnPayout = ClickPesa::lipaNambaPayouts()->create([
    'amount'      => 25000,
    'currency'     => 'TZS',
    'orderReference' => 'LN-901',
    'lipaNamba'    => '48001268',
    'providerCode' => '503',
]);
```

7. Webhooks & Event Processing

ClickPesa sends asynchronous HTTP POST notifications when transactions complete, fail, or reverse.

Setting Up Webhook Route in Laravel

Register the route macro in `routes/api.php` :

```
use Illuminate\Support\Facades\Route;

// Registers POST /clickpesa/webhooks
Route::clickpesaWebhooks('clickpesa/webhooks');
```

Listening to Dispatched Laravel Events

ClickPesa Event Name	Dispatched Laravel Event	Key Attributes
PAYMENT RECEIVED	ClickPesa\Laravel\Events\PaymentReceived	getOrderReference() , getCollectedAmount() , getCustomer()
PAYMENT FAILED	ClickPesa\Laravel\Events\PaymentFailed	getOrderReference() , getMessage()
PAYOUT INITIATED	ClickPesa\Laravel\Events\PayoutInitiated	getOrderReference() , getAmount() , getFee()
PAYOUT REFUNDED	ClickPesa\Laravel\Events\PayoutRefunded	getOrderReference() , getRefundMessage()
PAYOUT REVERSED	ClickPesa\Laravel\Events\PayoutReversed	getOrderReference() , getReverseMessage()
DEPOSIT RECEIVED	ClickPesa\Laravel\Events\DepositReceived	getDepositAmount() , getPaymentReference()

Example Listener Implementation

```
namespace App\Listeners;

use ClickPesa\Laravel\Events\PaymentReceived;

class MarkOrderAsPaid
{
    public function handle(PaymentReceived $event): void
    {
        $orderReference = $event->getOrderReference();
        $amount = $event->getCollectedAmount();
        $paymentRef = $event->getPaymentReference();

        // Update database order status
        $order = Order::where('reference', $orderReference)->first();
        if ($order) {
            $order->update([
                'status' => 'paid',
                'clickpesa_payment_reference' => $paymentRef,
            ]);
        }
    }
}
```

8. Standalone PHP & Application Testing

Using the SDK in Vanilla PHP / Non-Laravel Projects

```
require_once __DIR__ . '/vendor/autoload.php';

use ClickPesa\ClickPesaClient;

$client = new ClickPesaClient([
    'client_id'      => 'YOUR_CLIENT_ID',
    'api_key'        => 'YOUR_API_KEY',
    'checksum_key'   => 'YOUR_CHECKSUM_KEY',
    'checksum_enabled' => true,
]);

// Initiate USSD-PUSH
$res = $client->ussdPush()->initiate([
    'amount'         => '10000',
    'currency'        => 'TZS',
    'orderReference' => 'PHP_ORD_1',
    'phoneNumber'    => '255712345678',
]);
```

Testing & Mocking with `ClickPesa::fake()`

The SDK includes a first-class fake mechanism so your tests never make live network calls:

```

use ClickPesa\Laravel\Facades\ClickPesa;
use Tests\TestCase;

class CheckoutTest extends TestCase
{
    public function test_user_can_checkout(): void
    {
        // Mock all ClickPesa API calls
        ClickPesa::fake();

        $response = $this->postJson('/checkout', [
            'amount' => 5000,
            'phone' => '255712345678',
        ]);

        $response->assertOk();

        // Assert payment call occurred with expected parameters
        ClickPesa::assertInitiatedUssdPush(function ($data) {
            return $data['amount'] === '5000' && $data['phoneNumber'] === '255712345678';
        });

        // Assert no unauthorized payouts were triggered
        ClickPesa::assertNotSent('/payouts/create-bank-payout');
    }
}

```

9. Authors, Support & Commercial Inquiries



UBITECH SOLUTIONS LIMITED

We build custom payment gateways, enterprise fintech solutions, ERP integrations, and scalable mobile/web platforms across East Africa.

 **Telephone / WhatsApp: +255 766 192 332**

 **GitHub Organization: github.com/ubitechsolutionsltd**

 **Package Repository: [clickpesa-laravel-package](#)**

Licensed under the MIT License. © 2026 UBITECH SOLUTIONS LIMITED. All rights reserved.